

# Network Scanner Scripts

Pulling, running, and configuring `network_scanner.py` and `mobile_network_scanner.py`

---

Repository: <https://github.com/thomasmd321/Test-repo>

Branch: `claude/local-network-device-discovery-joawq5`

Both scripts discover devices on your local network with no external service and no account required. **network\_scanner.py** targets a normal desktop/Termux environment and can read ARP/MAC data directly; **mobile\_network\_scanner.py** targets a sandboxed environment such as an iPhone running a-Shell, where the OS blocks raw sockets and subprocess calls, so it relies on plain TCP connections instead.

|                              | <code>network_scanner.py</code>           | <code>mobile_network_scanner.py</code>    |
|------------------------------|-------------------------------------------|-------------------------------------------|
| Target platform              | Linux / macOS / Windows / Termux          | iOS sandboxes (a-Shell, Pythonista)       |
| Discovery method             | ARP scan (scapy), ping-sweep fallback     | Plain TCP connect probe                   |
| MAC address / vendor         | Yes (IEEE OUI lookup)                     | No (not exposed to the sandbox)           |
| IPv6 discovery               | Yes ( <code>--ipv6</code> )               | No                                        |
| Banner grabbing              | <code>--identify IP deep dive</code>      | Built into every scan                     |
| Risky-port flagging          | Yes                                       | Yes                                       |
| NEW / CHG / missing tracking | Yes                                       | Yes                                       |
| Export results               | <code>--output FILE (CSV/JSON)</code>     | <code>--output FILE (CSV/JSON)</code>     |
| Custom device labels         | <code>--set-label / --remove-label</code> | <code>--set-label / --remove-label</code> |

# 1. Getting the scripts

Both scripts are plain, dependency-light Python files living in one repository. Pick whichever pull method fits where you're running them.

## Clone the whole repository (desktop / Termux)

```
git clone https://github.com/thomasmd321/Test-repo.git
cd Test-repo
git checkout claude/local-network-device-discovery-joawq5
```

## Grab a single file directly (no git needed)

Useful on iOS, or anywhere you only want one script without the rest of the repo:

```
BASE=https://raw.githubusercontent.com/thomasmd321/Test-repo/claude/local-network-device-discovery-joawq5
curl -O $BASE/network_scanner.py
curl -O $BASE/mobile_network_scanner.py
```

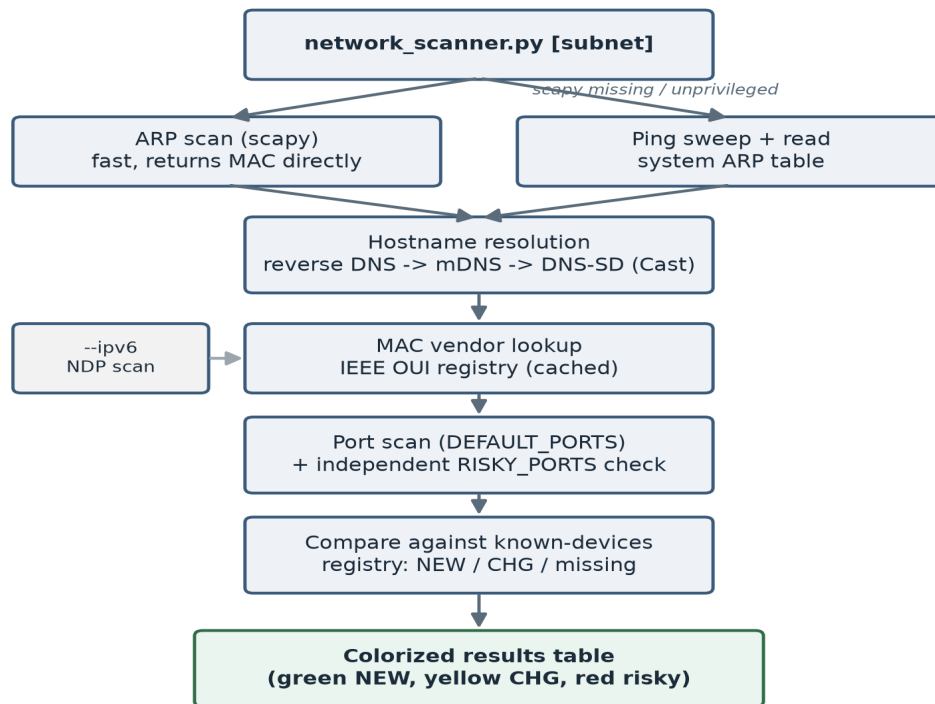
`curl -O` saves the file under its own name in the current directory and overwrites an existing copy, so re-running the same command later pulls in any updates.

## On an iPhone

Install **a-Shell** from the App Store (not “a-Shell mini,” which strips out `git`), then run either the clone or the single-file `curl` command above from its terminal.

## 2. network\_scanner.py

For a normal desktop, laptop, or Termux-on-Android environment. Prefers a fast ARP scan via scapy (returns MAC addresses directly); if scapy isn't installed or the process lacks the privileges ARP scanning needs, it falls back automatically to a multithreaded ping sweep plus the system's own ARP table — no flag needed to trigger this, it just happens.



*Falls back automatically when scapy / raw sockets aren't available.*

*How a scan flows through network\_scanner.py, start to finish.*

## Running it

```
python3 network_scanner.py                # auto-detect subnet
python3 network_scanner.py 192.168.1.0/24  # scan a specific subnet
python3 network_scanner.py --all-subnets   # every subnet (needs psutil)
python3 network_scanner.py --identify 192.168.1.42 # deep-dive one host
python3 network_scanner.py --watch 300      # rescan every 5 minutes
python3 network_scanner.py --output scan.json # save results to a file
```

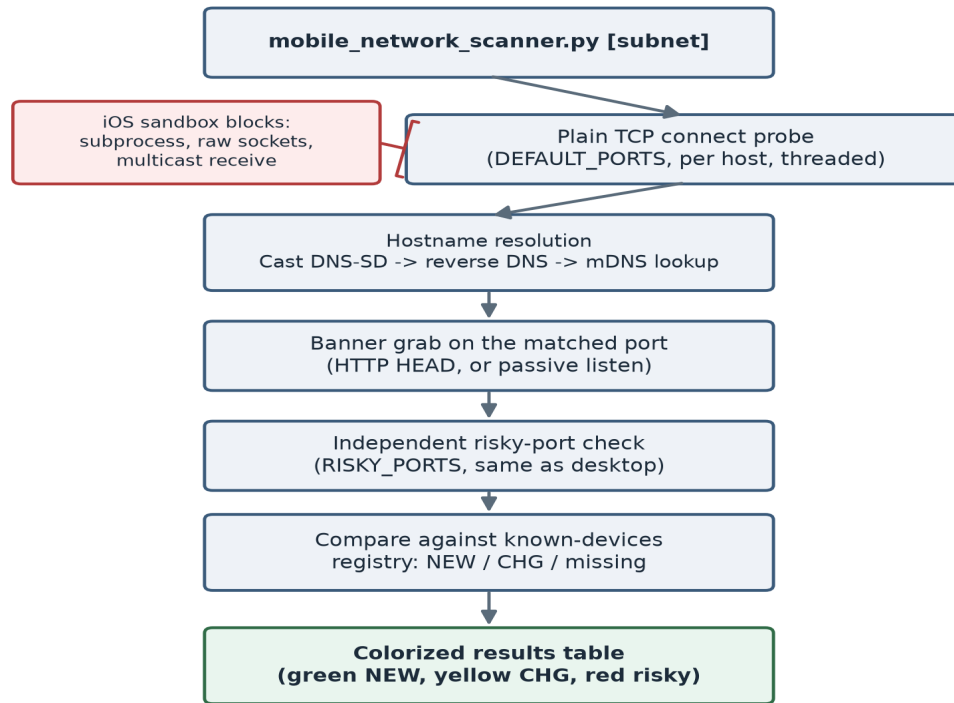
## Options

| Flag                 | What it does                                                                                                                            |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>subnet</b>        | Positional. CIDR range(s) to scan, comma-separated for more than one. Auto-detected if omitted.                                         |
| <b>--all-subnets</b> | Scan every local subnet this machine has an interface on (requires <code>pip install psutil</code> ) instead of just the default route. |

| Flag                          | What it does                                                                                                                              |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>--identify IP</b>          | Skip the network scan; do a slow, thorough single-host investigation instead (more ports, banner grabs, full hostname/vendor resolution). |
| <b>--timeout SECONDS</b>      | Timeout per host (default: 1.0).                                                                                                          |
| <b>--mdns-timeout SECONDS</b> | Timeout for the mDNS/DNS-SD hostname fallback (default: 0.3).                                                                             |
| <b>--no-vendor-lookup</b>     | Skip the IEEE OUI vendor lookup (avoids the first-run registry download).                                                                 |
| <b>--refresh-vendor-db</b>    | Force a fresh OUI registry download instead of using the cached copy.                                                                     |
| <b>--watch SECONDS</b>        | Rescan on a timer instead of once, printing NEW markers as they appear (Ctrl+C to stop).                                                  |
| <b>--no-track-devices</b>     | Don't use the known-devices registry at all — no NEW/CHG markers, nothing remembered.                                                     |
| <b>--forget-known-devices</b> | Clear the registry first, so everything in this run shows as NEW.                                                                         |
| <b>--set-label KEY=LABEL</b>  | Assign a friendly label to a device (KEY is its MAC, or IP if it has none), shown instead of/alongside its hostname. Repeatable.          |
| <b>--remove-label KEY</b>     | Remove a device's custom label. Repeatable.                                                                                               |
| <b>--ipv6</b>                 | Also discover IPv6 devices via multicast ping + NDP (Linux/macOS only).                                                                   |
| <b>--ipv6-timeout SECONDS</b> | Roughly how long to spend on IPv6 discovery (default: 2.0).                                                                               |
| <b>--no-scan-ports</b>        | Skip the open-port probe (and the risky-ports check that depends on it).                                                                  |
| <b>--ports LIST</b>           | Comma-separated TCP ports to probe instead of the built-in default list.                                                                  |
| <b>--port-timeout SECONDS</b> | Per-port connection timeout, for both the port probe and risky-ports check (default: 0.3).                                                |
| <b>--no-risky-ports</b>       | Skip the risky-ports security check while keeping the general port probe.                                                                 |
| <b>--no-color</b>             | Disable ANSI color output (also respects the NO_COLOR env var).                                                                           |
| <b>--output FILE</b>          | Save this scan's results to FILE as JSON, or CSV if it ends in .csv.                                                                      |

### 3. mobile\_network\_scanner.py

For sandboxed Python runtimes — most notably iOS apps like a-Shell — that allow neither subprocess calls nor raw sockets. Every discovery technique here is built from ordinary client TCP/UDP sockets, which is the one thing every one of these sandboxes still permits.



*No ARP, no IPv6 discovery, no MAC vendor lookup - ordinary sockets only.*

*How a scan flows through mobile\_network\_scanner.py, start to finish.*

### Running it

```
python3 mobile_network_scanner.py           # auto-detect subnet
python3 mobile_network_scanner.py 192.168.1.0/24 # scan a specific subnet
python3 mobile_network_scanner.py --timeout 0.5 --ports 22,80,443
python3 mobile_network_scanner.py --watch 300      # rescan every 5 minutes
python3 mobile_network_scanner.py --no-banners --no-risky-ports # faster scan
python3 mobile_network_scanner.py --output scan.csv # save results to a file
```

### Options

| Flag                     | What it does                                                                                    |
|--------------------------|-------------------------------------------------------------------------------------------------|
| <b>subnet</b>            | Positional. CIDR range(s) to scan, comma-separated for more than one. Auto-detected if omitted. |
| <b>--timeout SECONDS</b> | Timeout per port probe (default: 0.5); also used for the banner-grab and risky-ports steps.     |
| <b>--ports LIST</b>      | Comma-separated TCP ports to probe instead of the built-in default list.                        |

| Flag                          | What it does                                                                                                      |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>--mdns-timeout SECONDS</b> | Timeout for the mDNS/Bonjour hostname fallback (default: 0.3).                                                    |
| <b>--watch SECONDS</b>        | Rescan on a timer instead of once, printing NEW markers as they appear (Ctrl+C to stop).                          |
| <b>--no-track-devices</b>     | Don't use the known-devices registry at all — no NEW/CHG markers, nothing remembered.                             |
| <b>--forget-known-devices</b> | Clear the registry first, so everything in this run shows as NEW.                                                 |
| <b>--set-label KEY=LABEL</b>  | Assign a friendly label to a device (KEY is its IP address), shown instead of/alongside its hostname. Repeatable. |
| <b>--remove-label KEY</b>     | Remove a device's custom label. Repeatable.                                                                       |
| <b>--no-banners</b>           | Skip banner grabbing on each device's open port — faster, but a weaker identification hint.                       |
| <b>--no-risky-ports</b>       | Skip the risky-ports security check while keeping the general port probe.                                         |
| <b>--no-color</b>             | Disable ANSI color output (also respects the NO_COLOR env var).                                                   |
| <b>--output FILE</b>          | Save this scan's results to FILE as JSON, or CSV if it ends in .csv.                                              |

**iOS limitation:** mDNS/DNS-SD queries fail outright on iOS with `0SError(65, 'No route to host')` — Apple's Local Network Privacy model requires an app bundle to declare Bonjour service types in advance, which a script run from a generic terminal app can't do. Plain TCP port scanning, banner grabbing, and risky-port checks are unaffected, since they're ordinary unicast traffic.

## 4. Known-device tracking, color coding, and --watch

Both scripts persist a small local registry of every device they've ever seen

(`~/.cache/network_scanner_known_devices.json` and

`~/.cache/mobile_network_scanner_known_devices.json`). Every scan is compared against it, producing the markers and colors below. Pair either script with `--watch SECONDS` to turn a one-shot scan into a standing monitor.

```
python3 network_scanner.py

Scanning 192.168.1.0/24 ...

  IP Address      MAC Address      Vendor      Port  Service  Hostname
-----
NEW 192.168.1.42   3c:22:fb:1a:9c:04 Apple, Inc  62078 lockdownd iPhone.local
192.168.1.1      aa:bb:cc:dd:ee:ff Netgear    80      http     router.local
CHG 192.168.1.57   10:2c:6b:44:2a:11 Sonos      23      ?        kitchen-speaker.local
192.168.1.72     b0:c5:54:2f:88:19 Google     8009    chromecast Living Room TV
192.168.1.88     f4:5e:ab:12:34:56 Synology   445     smb      nas.local      (risky)

5 device(s) found. 1 new since last seen.
2 previously-seen device(s) not found in this scan:
192.168.1.63      last seen 2026-09-14T08:03:00 (laptop.local)
1 device(s) with a changed port since last seen:
192.168.1.57      (none) -> telnet (23)
1 device(s) exposing commonly-risky ports:
192.168.1.88     f4:5e:ab:12:34:56 smb (445)
```

*Illustrative output — exact columns vary slightly between the two scripts.*

| Marker / color | Meaning                                                                     |
|----------------|-----------------------------------------------------------------------------|
| NEW (green)    | First time this device has ever been seen.                                  |
| CHG (yellow)   | A known device is answering on a different port than last time.             |
| red row        | Device exposes a port on the risky-ports list (telnet, SMB, RDP, VNC, FTP). |
| dim gray line  | A previously-seen device that didn't show up in this scan.                  |

A device can be NEW or CHG, never both — a device with no prior registry entry can't have a “changed” port, only a first one. Risky-row coloring takes priority over both if a device qualifies for more than one.

### Resetting tracking

```
python3 network_scanner.py --no-track-devices      # skip tracking this run
python3 network_scanner.py --forget-known-devices  # reset registry, all NEW
python3 mobile_network_scanner.py --watch 300      # standing monitor on phone
```

## Custom device labels

The same registry can hold a friendly label for a device — useful when its real hostname is cryptic or blank — shown in place of/alongside the hostname (e.g. Kitchen Server (localhost) when they differ). KEY is a device's MAC (or IP if it has none) for network\_scanner.py, or always its IP for mobile\_network\_scanner.py.

```
python3 network_scanner.py --set-label aa:bb:cc:dd:ee:ff="Kitchen Server"
python3 mobile_network_scanner.py --set-label 192.168.1.42="Kitchen Echo"
python3 network_scanner.py --remove-label aa:bb:cc:dd:ee:ff
```

Both flags are repeatable and take effect immediately, including on the scan that runs in the same command. A device doesn't need to already be in the registry — labeling one creates a minimal entry for it, though that also means it won't show as NEW the next time it's actually scanned.

## 5. Exporting results

Both scripts can save a scan's results to a file with `--output FILE`, independent of the known-devices registry above — useful for feeding results into another tool, diffing two scans, or just keeping a dated record. The format is chosen by the file extension: `.csv` writes a CSV file, anything else (typically `.json`) writes JSON.

```
python3 network_scanner.py --output scan.json
python3 network_scanner.py --output scan.csv
python3 mobile_network_scanner.py --output scan.json
```

Each run overwrites FILE with that scan's results — this is a snapshot of the latest scan, not an appended history log.